

File I/O

Gestione della persistenza

Lezione 17
Corso 1



File e file system

29-04-2026 17:11

- Il Sistema Operativo ci offre la capacità di eseguire processi e di **memorizzare in modo permanente delle informazioni**
- Un **file** consiste in un'astrazione offerta dal sistema operativo che lega **un blocco di byte** di dimensione arbitraria ad un nome
 - I file vivono in un'organizzazione chiamata file system con metadati che descrivono ciascun file (ad es. ogni file ha un nome, ha un possessore, dispone di diritti di accesso, data di creazione, data di ultima modifica, data di ultimo accesso)
 - I nomi sono organizzati in una struttura gerarchica fatta di cartelle o directory che permette di identificare uno specifico file attraverso una concatenazione dei nomi di cartelle e nome del file che ne esprime il cammino (*path*) a partire da una radice nota
- I file possono:
 - Essere scritti (creati da zero)
 - È possibile aggiungere informazioni in fondo (*append*)
 - Possono anche essere modificati al loro interno poiché l'accesso ai file è random (posizionandosi in qualunque posizione del file e leggerne e scrivere il contenuto).
- Ad ogni file sono associati vincoli di sicurezza
 - Il sistema operativo garantisce che solo chi dispone delle necessarie autorizzazione possa leggere, scrivere o eseguire il file
 - In Unix, si distinguono i diritti del possessore (user), del gruppo e degli altri utenti (bit map di 9 bit che descrivono i diritti, rappresentati attraverso 3 numeri ottali, ad es. 777 oppure 755 oppure 644 oppure 700 oppure 600)
- Le librerie dei linguaggi di programmazione offrono meccanismi indipendenti dal sistema operativo (cross-piattaforma) per accedere al contenuto di un file
 - Nel caso di Rust, l'astrazione principale è offerta dalla struct `std::fs::File`, che modella un **file** aperto in lettura e/o scrittura e ci offre anche il concetto di **cartella** dandoci la possibilità di elencare quello che c'è presente.

Percorsi

Path e PathBuf mi permette di far riferimento ad una serie di file o directory

- Ogni sistema operativo ha regole proprie per la definizione di cosa sia un percorso (*path*) lecito per indicare un file, quali siano le radici note dei percorsi, come combinare segmenti parziali in un percorso complessivo, ...
 - I tipi `std::path::Path` e `std::path::PathBuf` nascondono tali differenze offrendo un meccanismo portabile per comporre e scomporre un cammino e ricavare indicazioni sul file eventualmente referenziato
 - `Path`, analogamente a `str`, è *unsized* e accessibile in sola lettura
 - `PathBuf`, analogamente a `String`, possiede il proprio contenuto e può essere modificato
 - `Path` e `PathBuf` offrono proprietà diverse rispetto ai tipi `str` e `String` perché permettono navigare il file system di cartella in cartella, segmentando il path. Il problema è che il separatore è differente tra Windows (back slash `\`) e Unix (forward slash `/`).
- Attraverso i **metodi** offerti da questi tipi, è possibile ricavare informazioni sulla esistenza del file, sulla sua natura (file semplice, cartella, collegamento simbolico *symlink*, ...), sui metadati associati (dimensione, data di creazione e di ultima modifica, permessi, ...)

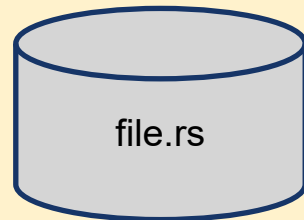
Navigare il file system

- La funzione `std::fs::read_dir(dir: &Path) -> Result<ReadDir>` restituisce, se ha successo, un iteratore al contenuto della cartella `dir`
 - Le singole voci ritornate sono di tipo `std::fs::DirEntry` e descrivono gli elementi contenuti nella cartella in termini di nome, tipo (file, cartella, collegamento simbolico), metadati e cammino
- La funzione `std::fs::create_dir(dir: &Path) -> Result<()>` crea una nuova cartella
 - Fallisce se non si dispone delle necessarie autorizzazioni, se la cartella esiste già o se la cartella genitrice del cammino indicato non esiste
- La funzione `std::fs::remove_dir(dir: &Path) -> Result<()>` rimuove una cartella
 - A condizione che esista, si disponga dei necessari permessi e che sia vuota

```

use std::fs;
use std::path::{Path, PathBuf};
fn main() -> std::io::Result<()> {
    let mut p = PathBuf::from("src");
    p.push("main.rs"); // aggiunge un segmento in modo portabile
    println!("Path completo: {:?}", p);
    let path: &Path = p.as_path();
    println!("Esiste? {}", path.exists());
    println!("È un file? {}", path.is_file());
    println!("È una directory? {}", path.is_dir());
    println!("Nome file: {:?}", path.file_name());
    println!("Estensione: {:?}", path.extension());
    println!("File stem: {:?}", path.file_stem());
    println!("Parent: {:?}", path.parent());
    // Itera sui componenti del path
    println!("Componenti:");
    for c in path.components() {
        println!("  {:?}", c);
    }
    if path.exists() {
        let metadata = fs::metadata(path)?; // segue eventuali symlink
        println!("Dimensione: {} byte", metadata.len());
        println!("Readonly? {}", metadata.permissions().readonly());
        if let Ok(modified) = metadata.modified() {
            println!("Ultima modifica: {:?}", modified);
        }
        if let Ok(created) = metadata.created() {
            println!("Creato il: {:?}", created);
        }
    }
    Ok(())
}

```



read_dir()

```
fn main() -> std::io::Result<()> {  
    // Ottieni il percorso della directory  
    let directory_path = ".";  
  
    // Leggi il contenuto della directory  
    let entries = fs::read_dir(directory_path)?;  
  
    // Itera sugli elementi nella directory  
    for entry in entries {  
        // Gestisci eventuali errori nell'accesso ai file/directory  
        let entry = entry?;  
  
        // Ottieni il nome dell'elemento  
        let file_name = entry.file_name();  
  
        // Stampa il nome dell'elemento  
        println!("{:?}", file_name);  
    }  
  
    Ok(())  
}
```

read_dir crea un iteratore



il ? indica l'operatore try, in quanto è possibile che possa avvenire un errore nell'accesso della directory

create_dir()

```
use std::fs;

fn main() -> std::io::Result<> {
    // Definisci il percorso della nuova directory da creare
    let new_directory_path = "./mynewdir";

    // Crea la nuova directory
    fs::create_dir(new_directory_path)?;

    println!("Directory creata con successo!");

    Ok(())
}
```



remove_dir()

```
use std::fs;

fn main() -> std::io::Result<()> {
    // Definisci il percorso della directory da rimuovere
    let directory_to_remove = "./mynewdir";

    // Rimuovi la directory
    fs::remove_dir(directory_to_remove)?;

    println!("Directory rimossa con successo!");

    Ok(())
}
```



Manipolare i file nel file system

- La funzione `std::fs::copy(from: &Path, to: &Path) -> Result<i64>` copia il contenuto di un file in un secondo file
 - Restituisce in caso di successo il numero di byte copiati
- La funzione `std::fs::rename(from: &Path, to: &Path) -> Result<()>` rinomina (o sposta) un file in un secondo file
 - Sostituendo il contenuto del file destinazione con quello sorgente
 - Il comportamento di questa funzione dipende dal sistema operativo
- La funzione `std::fs::remove_file(path: &Path) -> Result<()>` elimina un file
 - Se il file è in uso, la sua eliminazione può essere rimandata dal sistema operativo

copy()

```
use std::fs;
use std::io;

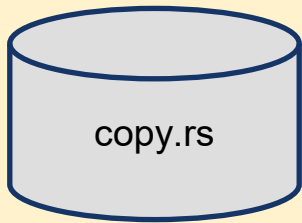
fn main() -> io::Result<()> {
    // Percorso del file di origine
    let source_path = "./prova.txt";

    // Percorso di destinazione per il file copiato
    let destination_path = "./file.txt";

    // Copia il file
    fs::copy(source_path, destination_path)?;

    println!("File copiato con successo!");

    Ok(())
}
```



rename()

```
use std::fs;

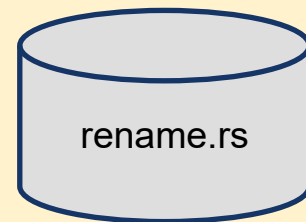
fn main() -> std::io::Result<()> {
    // Definisci il percorso del file o della directory da rinominare
    let old_path = "./file.txt";

    // Definisci il nuovo nome o percorso del file o della directory
    let new_path = "./new.txt";

    // Rinomina il file o la directory
    fs::rename(old_path, new_path)?;

    println!("Rinominato con successo!");

    Ok(())
}
```



remove_file()

```
use std::fs;

fn main() -> std::io::Result<()> {
    // Definisci il percorso del file da rimuovere
    let file_to_remove = "./new.txt";

    // Rimuovi il file
    fs::remove_file(file_to_remove)?;

    println!("File rimosso con successo!");

    Ok(())
}
```



Operazioni con i file

- L'accesso al blocco di byte legato ad un file è totalmente mediato dal sistema operativo
 - Per poter leggere o scrivere tale blocco occorre “aprire” il file
 - Il sistema operativo offre apposite funzioni che restituiscono un riferimento opaco al file sotto forma di *handle* o *file descriptor* (di fatto un numero intero)
- La struct **File** offre due metodi di base per aprire un file
 - **open(path: P) -> Result<File> where P: AsRef<Path>** - apre il file in lettura, a condizione che esista
 - **create(path: P) -> Result<File> where P: AsRef<Path>** - tronca il file a 0 byte, se esiste, o lo crea, se non esiste ancora, dopodiché lo apre in scrittura
- P deve godere del tratto **AsRef<Path>**
 - Il tratto **AsRef<Path>** è implementato dai tipi che possono essere convertiti in un riferimento a **Path** (come **String** o **&str**).

il nome del file, tradizionalmente una stringa, in Rust ha una rappresentazione ad hoc. Rust definisce un tipo specifico per rappresentare un path; questo tipo prende nome di `AsRef<Path>`. Il nome del file scritto tra doppi apici non sarebbe corretto; il compilatore lo converte in automatico rispetto al `Path`. La `string` e la `str` godono del tipo `Path`, e quindi vengono convertiti. Però dobbiamo sapere che nell'implementazione di Rust, c'è un tipo diverso, utile per garantire compatibilità tra i sistemi operativi, in cui in qualcuno si usa il backslash, in altri si usa frontslash

```
use std::fs::File;
use std::io::prelude::*;
use std::io::Error;

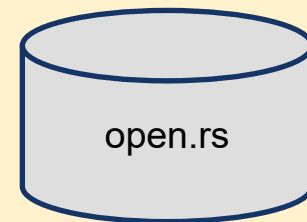
fn main() -> Result<(), Error> {
    // Definisci il percorso del file da aprire
    let file_path = "./myfile";

    // Apri il file in modalità di lettura
    let mut file = File::open(file_path)?;

    // Leggi il contenuto del file in una stringa
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;

    // Stampa il contenuto del file
    println!("Contenuto del file:");
    println!("{}", contents);

    Ok(())
}
```



```
use std::fs::File;
use std::io::prelude::*;
use std::io::Error;

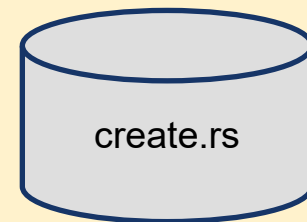
fn main() -> Result<(), Error> {
    // Definisci il percorso del nuovo file da creare
    let file_path = "./new_file.txt";

    // Crea un nuovo file
    let mut file = File::create(file_path)?;

    // Scrivi una stringa direttamente nel file
    let text = "Questo è un nuovo file creato in Rust!";
    file.write(text.as_bytes())?;

    println!("File creato e scritto con successo!");

    Ok(())
}
```



Operazioni con i file (II)

- Maggiori opportunità sono offerte dalla struct **`std::fs::OpenOption`**
 - viene utilizzata per configurare le opzioni di apertura di un file prima di aprirlo effettivamente tramite la funzione **`open`**
- **`OpenOptions`** fornisce diversi metodi per configurare queste opzioni, tra cui:
 - **`.read(bool)`**: imposta la modalità di apertura per la lettura del file
 - **`.write(bool)`**: imposta la modalità di apertura per la scrittura del file
 - **`.create(bool)`**: specifica se creare il file se non esiste
 - **`.truncate(bool)`**: specifica se troncare il file se esiste già
 - **`.append(bool)`**: specifica se scrivere alla fine del file invece di sovrascrivere il contenuto esistente.

i metodi ricevono un booleano, per specificare la modalità di apertura del file

```
use std::fs::OpenOptions;  
use std::io::prelude::*;
```

```
fn main() -> std::io::Result<()> {  
    let file_path = "./new_file.txt";
```

```
    // Aprire il file in modalità di scrittura, tronendolo se esiste già  
    let mut file = OpenOptions::new()  
        .write(true)  
        .truncate(true)  
        .open(file_path)?;
```

nota che la open è diversa da quella definita nella slide 13. In questo codice la open viene applicata ad un ricevitore, c'è un parametro self, nel senso che sto applicando il metodo ad una variabile di tipo OpenOptions.

```
    let text = "Questo è un nuovo file creato in Rust!";  
    file.write(text.as_bytes())?;
```

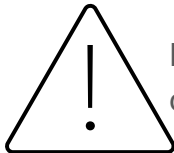
```
    Ok(())
```

```
}
```



Leggere e scrivere file

- `std::fs::read_to_string(path: &Path) -> Result<String, std::io::Error>`
`std::fs::write(path: &Path, contents: &[u8]) -> Result<(), std::io::Error>`
- offrono un meccanismo compatto per leggere e scrivere il contenuto di un file di testo di moderate dimensioni
 - `read_to_string()` legge l'intero contenuto di un file e lo mappa in una stringa
 - *Poiché un file può avere dimensioni molto maggiori del massimo blocco di memoria allocabile, occorre utilizzare tale funzione quando si è certi che il contenuto può essere ospitato nella memoria del processo*
 - `write()` prende un byte array e lo scrive all'interno di un file.



Da non confondersi con le funzioni con lo stesso nome che vedremo dopo. Guardando la signature, si vede la profonda differenza.

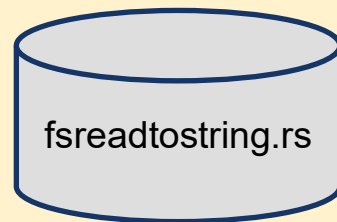
```
use std::fs;
use std::io::Error;

fn main() -> Result<(), Error> {
    let file_path = "./file.txt";

    // Leggi il contenuto del file in una stringa
    let contents = fs::read_to_string(file_path)
        .expect("Something went wrong reading the file");

    // Stampare il contenuto del file
    println!("Contenuto del file:\n{}", contents);

    Ok(())
}
```



```
use std::fs;
use std::io::Error;

fn main() -> Result<(), Error> {
    let file_path = "./file.txt";
    let text = "Questo è un testo scritto con fs::write!";
    fs::write(file_path, text)?;

    Ok(())
}
```

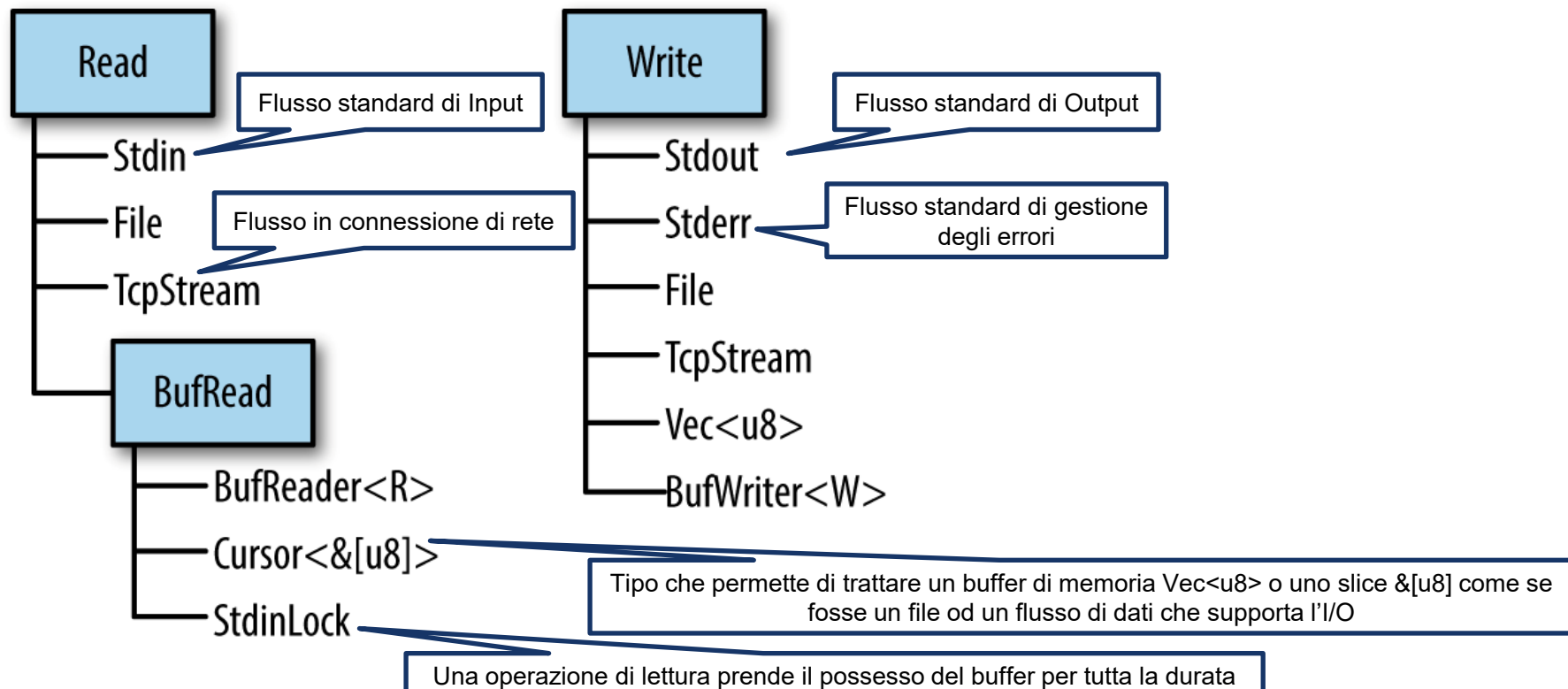


I tratti relativi a I/O

11:53 04-05-2026

- Rust gestisce le operazioni di I/O attraverso l'utilizzo di alcuni tratti che implementano i metodi di base per le attività di lettura e scrittura
 - L'utilizzo di tratti favorisce la scrittura di codice generico, indipendente dal tipo specifico su cui viene eseguito
- I tratti principali offerti da Rust sono: **Read**, **BufRead**, **Write** e **Seek**

I tratti relativi ad I/O



Enum ErrorKind

- In caso di errore durante le operazioni di I/O viene ritornata una delle varianti disponibili nell'enum **ErrorKind** che rappresenta le diverse categorie di errori che possono verificarsi durante le operazioni di input/output (I/O). Questi errori sono suddivisi in categorie per facilitare la loro gestione e consentire una maggiore precisione nell'individuazione dei problemi. Ecco un elenco di alcune varianti di **ErrorKind**:
 - **NotFound**: Indica che il file o la directory specificati non sono stati trovati.
 - **PermissionDenied**: Indica che non si dispone delle autorizzazioni necessarie per eseguire l'operazione.
 - **AlreadyExists**: Indica che il file o la directory che si sta cercando di creare esiste già.
 - **InvalidInput**: Indica che l'input fornito è invalido o non valido per l'operazione.
 - **TimedOut**: Indica che l'operazione ha raggiunto il timeout specificato.
 - **Interrupted**: Indica che l'operazione è stata interrotta (ad esempio, da un segnale di interruzione), indica un errore non fatale che generalmente può essere gestito riprovando l'operazione.


```
use std::io::{ErrorKind, Read};
```

11:58

```
use std::fs::File;
```

```
fn main() {
```

```
    // Tentativo di apertura di un file
```

```
    match File::open("testo.txt") {
```

```
        Ok(mut file) => {
```

```
            let mut contenuto = String::new();
```

```
            // Tentativo di lettura del contenuto del file
```

```
            match file.read_to_string(&mut contenuto) {
```

```
                Ok(_) => println!("Contenuto del file: {}", contenuto),
```

questa funzione

read_to_string gode del

tratto Read, ed è quindi

diversa

dalla funzione definita

nella

slide 18

```
                Err(e) => match e.kind() {
```

```
                    // Gestione specifica per diversi tipi di errori
```

```
                    ErrorKind::NotFound => println!("Il file non è stato trovato."),
```

```
                    ErrorKind::PermissionDenied => println!("Permesso negato."),
```

```
                    _ => println!("Si è verificato un errore durante la lettura del file: {}", e),
```

```
                },
```

```
            }
```

```
        },
```

```
        Err(e) => match e.kind() {
```

```
            ErrorKind::NotFound => println!("Il file non è stato trovato."),
```

```
            ErrorKind::PermissionDenied => println!("Permesso negato."),
```

```
            _ => println!("Si è verificato un errore durante l'apertura del file: {}", e),
```

```
        },
```

```
    }
```

```
}
```

il metodo kind applicato
su "e" mi permette di esplicitare
le varianti



std::io::Read

importante ricordare che `read` riceve come parametro un array di byte (`&[u8]`). Essendo un array ha una sua dimensione, è una struttura dati con dimensione definita. pertanto dobbiamo precedentemente creare l'array (ovvero il buffer). Poi, tramite il metodo `read`, NOI STIAMO RIEMPIENDO IL BUFFER. Il `read` restituisce un `result` che può essere `Ok` oppure `Err`. `n` può essere al massimo pari alla lunghezza del buffer

- **Tratto che indica la capacità di leggere un flusso di byte**
 - `File`, `Stdin` e `TcpStream` sono alcuni dei tipi che implementano questo tratto
- Per implementare il tratto `Read` è sufficiente fornire l'implementazione del metodo `read(buf: &mut [u8]) -> Result<usize>`
 - Rust genererà tutti gli altri metodi sulla base dell'implementazione di `read(buf: &mut [u8])` fornita dal programmatore
- Il metodo `read()` legge un numero noto (pari alla dimensione del buffer) di byte alla volta
- In caso di successo, il metodo `read(...)` deve ritornare `Ok(n)`
 - Dove `n` rappresenta il numero di byte letti con successo nel buffer
 - L'implementazione deve garantire che `n` sia compreso tra 0 e `buf.len()`
 - Il valore `Ok(0)` può indicare che il flusso è terminato (end-of-file), oppure che il buffer passato ha lunghezza 0...
- Ogni chiamata al metodo `read(...)` può causare l'invocazione di una chiamata di sistema, con il conseguente costo legato al cambio di contesto

Text

```

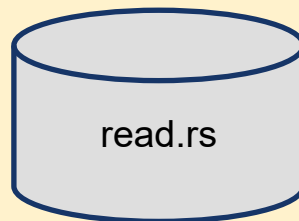
fn main() -> std::io::Result<()> {
let mut file = match File::open("divinacommedia.txt") {
    Ok(file) => file,
    Err(error) => {
        println!("Errore durante l'apertura del file: {}", error);
        return Err(error); }
};
let mut buffer = [0; 10]; // Crea un buffer di 10 byte per leggere i dati
let result = file.read(&mut buffer); // Tenta di leggere fino a 10 byte dal file nel buffer
match result {
    Ok(bytes_letti) => {
        println!("Sono stati letti {} byte dal file.", bytes_letti);
        if bytes_letti > 0 {
            let s = String::from_utf8_lossy(&buffer[..bytes_letti]);
            println!("Dati letti: {}", s);
        } else { println!("Fine del file raggiunto."); }
    }
    Err(e) => { eprintln!("Si è verificato un errore durante la lettura dal file: {}", e); }
}

let mut another_buffer = [0; 5]; // Tentiamo di leggere altri dati (dopo la prima lettura)
let second_result = file.read(&mut another_buffer);
match second_result {
    Ok(second_bytes_letti) => {
        println!("Sono stati letti altri {} byte dal file.", second_bytes_letti);
        if second_bytes_letti > 0 {
            let s = String::from_utf8_lossy(&another_buffer[..second_bytes_letti]);
            println!("Altri dati letti: {}", s);
        } else { println!("Fine del file raggiunto."); }
    }
    Err(e) => { eprintln!("Si è verificato un errore durante la seconda lettura dal file: {}", e); }
}

Ok(())
}

```

chiamo la read e le passo il buffer. Con questo dico di leggere al massimo 10 byte, in quanto in questo esempio il buffer ha dimensione 10



Metodi del tratto Read

- L'implementazione del tratto **Read** mette a disposizione diversi metodi per gestire le operazioni più comuni di I/O
 - **read_to_end(buf: &mut Vec<u8>) -> Result<usize>** continua a leggere fino all' EOF: si limita a richiamare il metodo **read()** fino a quando quest'ultimo non ritorna un **Ok(0)** o un errore fatale
 - **read_to_string(buf: &mut String) -> Result<usize>** continua a leggere fino all' EOF e riceve come parametro una **&mut String**
 - **read_exact(buf: &mut [u8]) -> Result<()>** prova a leggere l'esatto numero di byte necessario a riempire completamente **buf**, se non riesce, perché di dimensione inferiore, ritorna **ErrorKind::UnexpectedEof**
 - **bytes() -> Bytes<Self>** ritorna un iteratore sui bytes, gli elementi dell'iteratore sono dei **Result<u8, io::Error>**
 - **chain<R: Read>(next: R) -> Chain<Self, R>** permette di concatenare due reader
 - **take(limit: u64) -> Take<Self>** limita il numero massimo di byte che sarà possibile leggere

read_to_end() vs. read_to_string()

- `read_to_end()`:
 - Legge tutti i byte disponibili dalla sorgente fino alla fine e li accoda a un buffer di tipo `Vec<u8>` (un vettore di byte).
 - Mantiene i dati letti come una sequenza di byte non interpretata. È la funzione più generica per leggere dati binari o testuali.
- `read_to_string()`:
 - Legge tutti i byte disponibili dalla sorgente fino alla fine e tenta di **interpretarli** come una stringa UTF-8 valida, accodandoli a un buffer di tipo `String`
 - Assume che i dati in ingresso siano codificati in UTF-8. Se incontra byte non validi, l'operazione fallisce.

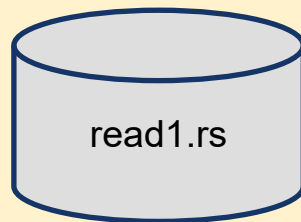
```

use std::fs::File;
use std::io::Read;
fn main() {
    // Apro il file in modalità lettura
    let mut file = match File::open("test.txt") {
        Ok(file) => file,
        Err(e) => {
            println!("Errore durante l'apertura del file: {}", e);
            return;
        }
    };

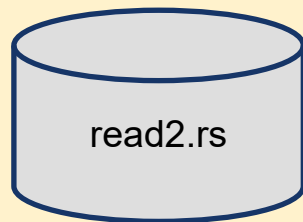
    // Creo un buffer vuoto per contenere i dati letti
    let mut buffer = Vec::new();

    // Leggo il contenuto del file nel buffer
    match file.read_to_end(&mut buffer) {
        Ok(_) => {
            // Converto il buffer in una stringa UTF-8 e stampo il contenuto
            match String::from_utf8(buffer) {
                Ok(content) => println!("{}", content),
                Err(_) => println!("Errore nella decodifica del contenuto del file"),
            }
        }
        Err(e) => println!("Errore durante la lettura del file: {}", e),
    }
}

```

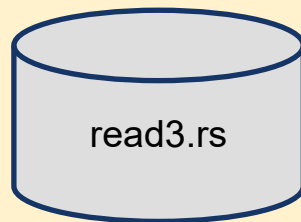


```
fn main() {  
    // Apro il file in modalità lettura  
    let mut file = match File::open("test.txt") {  
        Ok(file) => file,  
        Err(e) => {  
            println!("Errore durante l'apertura del file: {}", e);  
            return;  
        }  
    };  
  
    // Dichiaro una stringa vuota per contenere il contenuto del file  
    let mut content = String::new();  
  
    // Leggo il contenuto del file nella stringa  
    match file.read_to_string(&mut content) {  
        Ok(_) => {  
            // Stampo il contenuto del file  
            println!("{}", content);  
        }  
        Err(e) => println!("Errore durante la lettura del file: {}", e),  
    }  
}
```



```
fn main() -> io::Result<()> {  
    // Apro il file in modalità lettura  
    let file_path = "test.txt";  
    let text = "Ciao mamma guarda come mi diverto con Rust";  
    fs::write(file_path, text)?;  
  
    let mut file = File::open("test.txt"?);  
  
    // Dichiaro un array vuoto per contenere i byte letti  
    let mut buffer = [0; 30]; // Predispongo un buffer di 30 byte  
  
    file.read_exact(&mut buffer)?; // Leggo esattamente 30 byte dal file  
  
    println!("I byte letti sono: {:?}", String::from_utf8_lossy(&buffer));  
  
    let result = file.read_exact(&mut buffer); // Provo a leggere altri 30 byte  
  
    match result {  
        Ok(_) => println!("I byte letti sono: {:?}", String::from_utf8_lossy(&buffer)),  
        Err(_) => println!("Errore nella lettura dei byte: non ce n'erano altri 30"),  
    }  
    Ok(())  
}
```

guarda il codice dal portale perchè in questa slide mancano gli import dei moduli



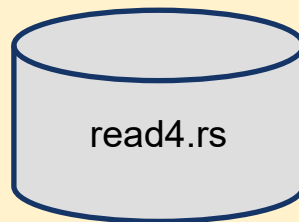
IMPORTANTE : se guardi il codice nella slide precedente noti che per trasformare il buffer è stato utilizzato un metodo che viene implementato dal tipo String. Quindi capisci che nel codice è stato usato questo metodo perchè l'obiettivo è ricavare una stringa

```
use std::fs::File;
use std::io::{self, Read};

fn main() -> io::Result<()> {
    // Apro il file in modalità lettura
    let mut file = File::open("test.txt"?);

    // Ottengo un iteratore sui byte del file
    // Itero sui byte e stampo il valore di ciascun byte
    for byte in file.bytes() {
        match byte {
            Ok(b) => println!("Byte: {} {}", b, char::from_u32(b as u32).unwrap()),
            Err(e) => println!("Errore durante la lettura del byte: {}", e),
        }
    }

    Ok(())
}
```



```

use std::fs::File;
use std::io::{self, Read};

fn main() -> io::Result<()> {
    // Apro due file in modalità lettura
    let file1 = File::open("file1.txt")?;
    let file2 = File::open("file2.txt")?;

    // Concateno i due lettori
    let mut chained_reader = file1.chain(file2);

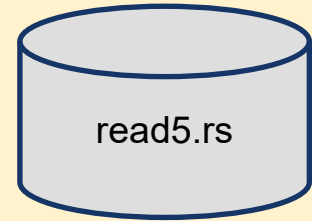
    // Dichiaro un buffer per contenere i dati letti
    let mut buffer = Vec::new();

    // Leggo i dati concatenati nei buffer
    chained_reader.read_to_end(&mut buffer)?;

    // Converto il buffer in una stringa UTF-8 e la stampo
    match String::from_utf8(buffer) {
        Ok(content) => println!("{}", content),
        Err(_) => println!("Errore nella decodifica del contenuto"),
    }

    Ok(())
}

```



```
use std::fs::File;
use std::io::{self, Read};
```

```
fn main() -> io::Result<()> {
    // Apro il file in modalità lettura
    let file = File::open("test.txt"?);
```

```
    // Creo un nuovo lettore che legge solo i primi 10 byte dal file originale
    let mut limited_reader = file.take(10);
```

```
    // Dichiaro un buffer per contenere i dati letti
    let mut buffer = Vec::new();
```

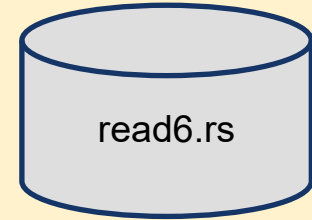
```
    // Leggo i primi 10 byte dal file e li memorizzo nel buffer
    limited_reader.read_to_end(&mut buffer)?;
```

```
    // Converto il buffer in una stringa UTF-8 e la stampo
    match String::from_utf8(buffer) {
        Ok(content) => println!("{}", content),
        Err(_) => println!("Errore nella decodifica del contenuto"),
    }
```

```
    Ok(())
```

```
}
```

un lettore è una variabile che implementa il tratto Read.
 file è un descrittore di file, ma poichè il tipo File implementa il tratto Read, lo possiamo chiamare lettore.
 In pratica sto creando un altro lettore e consumo il file originale



posso leggere solo da limited_read da questa istruzione in poi
 in quanto il metodo take consuma il file originale

std::io::BufRead

il tratto **BufRead** definisce un buffer interno → alla prima richiesta di lettura viene caricato un buffer di dimensione predefinita

- Sottotratto del tratto **Read**
- Il tratto **BufRead** offre una serie di metodi che permettono di migliorare le prestazioni dell'I/O appoggiandosi ad un buffer in memoria di grande dimensione (default: 8k byte)
 - Ogni chiamata a **read()** può dare origine ad una system call, l'utilizzo di un buffer permette di effettuare meno chiamate
 - Risulta particolarmente efficace se si eseguono molte letture di piccole dimensioni
 - Non è utile quando si legge da elementi già presenti in memoria
- L'implementazione del tratto richiede i metodi **fill_buf()** e **consume(amt: usize)**
 - **fill_buf()** ritorna il contenuto del buffer in memoria
 - **consume(...)** consuma il numero specificato di byte
- Offre i metodi **read_line(&mut self, buf: &mut String)** e **lines(self)** per accedere al contenuto testuale di un flusso

ad ogni chiamata della funzione **read()** di fa accesso ad una system call.
In questo caso la system call viene usata SOLO QUANDO IL BUFFER VIENE SVUOTATO

il tratto `BufRead` definisce un buffer interno —>
alla prima richiesta di lettura viene caricato un
buffer di dimensione predefinita

con `BufRead` l'utilizzo della system call viene fatto solo quando il buffer è stato completamente
letto

```
use std::fs::File;
use std::io;
use std::io::{Write, BufReader, BufRead};
```

```
fn main() -> io::Result<()> {
    let path = "myfile";
```

```
    let mut output = File::create(path)?;
    write!(output, "Rust\n💖\nFun")?;
```

```
    let input = File::open(path)?;
    let buffered = BufReader::new(input);
```

```
    for line in buffered.lines() {
        println!("{}", line?);
    }
```

```
    Ok(())
}
```

Creo un'istanza di bufreader



lines() restituisce un Result, se la linea è troppo grande e non può essere allocata ci ha dato un errore

```
use std::fs::File;
use std::io::{BufReader, Result, BufRead};

fn main() -> Result<()> {
    // Apre un file in lettura
    let file = File::open("example.txt"?);

    // Crea un BufReader con una dimensione del buffer di 1024 byte
    let mut buf_reader = BufReader::with_capacity(1024, file);

    // Esempio di utilizzo: legge una riga dal file
    let mut line = String::new();
    buf_reader.read_line(&mut line)?;
    println!("{}", line);

    Ok(())
}
```



- `read_line()` appende al buffer il contenuto letto
- Si suppone che il buffer sia vuoto

```
use std::fs::File;  
use std::io::{BufReader, Result, BufRead};
```

```
fn main() -> Result<()> {  
    // Apre un file in lettura  
    let file = File::open("example.txt");
```

```
    // Crea un BufReader con una dimensione del buffer di 1024 byte  
    let mut buf_reader = BufReader::with_capacity(1024, file);
```

```
    // legge una riga dal file  
    let mut line = String::new();  
    buf_reader.read_line(&mut line)?;  
    println!("{}", line);  
    line.clear();
```

è necessario fare un clear per resettare la struttura dati,
in quanto il metodo `.read_line()` fa un append della
riga appena letta nel buffer

```
    // legge una seconda riga dal file sulla stessa variabile buffer  
    buf_reader.read_line(&mut line)?;  
    println!("{}", line);  
    Ok(())
```

```
}
```

intendi la variabile `buf_reader` come uno
spazio in memoria inizialmente vuoto che
può essere usato per accedere ai dati
del file tramite i metodi, come ad esempio
`read_line`



fill_buf() e consume()

- Il metodo consume() è strettamente legato al metodo fill_buf().
- Il modello utilizzato tipico è:
 - Chiamare fill_buf() per ottenere un riferimento ai dati nel buffer interno
 - Elaborare i dati all'interno della slice restituita
 - Chiamare consume() per comunicare al BufReader quanti byte sono stati elaborati
- consume() è il meccanismo attraverso il quale si comunica al BufReader di aver utilizzato una porzione dei dati che ha fornito al suo buffer
- Senza consume() si finirebbe in un ciclo infinito, leggendo sempre gli stessi byte.

```

use std::fs::File;
use std::io::{BufReader, BufRead, Result};
use std::str;

fn main() -> Result<()> {
    let file = File::open("divinacommedia.txt")?;
    let mut reader = BufReader::new(file);
    let mut total_bytes_read = 0;
    loop {
        let buffer = reader.fill_buf()?;
        let len = buffer.len();

        if len == 0 {
            break; // Fine del file
        }
        match str::from_utf8(buffer) {
            Ok(s) => print!("{}", s),
            Err(_) => {
                eprintln!("Warning: Invalid UTF-8 encountered in buffer.");
            }
        }
        // Indica al BufReader quanti byte abbiamo letto (in questo caso, l'intero buffer).
        reader.consume(len);
        total_bytes_read += len;
    }
    println!("\nLettura completata. Totale byte letti: {}", total_bytes_read);
    Ok(())
}

```

12:25

guarda anche sul portale [bufreader4.rs](#)

il metodo `.fill_buf()` riempie il buffer
solo se esso è vuoto



std::io::Write

- **Tratto che indica la capacità di scrivere un flusso di dati**
 - Questo tratto è implementato, tra gli altri, dalle struct **File**, **Stdout**, **StdErr** e **TcpStream**
- Il tratto **Write** richiede l'implementazione dei metodi **write** e **flush**
 - **write(buf: &[u8]) -> Result<usize>** prova a scrivere l'intero contenuto del buffer ricevuto come argomento e ritorna il numero di byte scritti. Per questioni di ottimizzazione, il sistema operativo potrebbe scrivere solo parzialmente il contenuto del buffer ricevuto.
 - **flush() -> Result<()>** finalizza l'output garantendo che tutti gli eventuali buffer transitori siano correttamente svuotati
- Il metodo **write_all(buf: &[u8])** si limita a chiamare ricorsivamente il metodo **write** fino a quando i dati sono stati tutti scritti o viene restituito un errore fatale

bisogna fare attenzione ad utilizzare il metodo write, in quanto è possibile che non completi l'operazione di scrittura. Si suggerisce l'utilizzo di write_all

```

use std::io;
use std::fs::File;
use std::io::Write;
fn main() -> io::Result<()> {
    let mut file = File::create("output.txt"?);

    // Dati da scrivere nel file
    let data = b"Hello, world!\n";

    let num_writes = 5; // Numero di volte che voglio scrivere i dati nel file
    let mut total_bytes_written = 0;

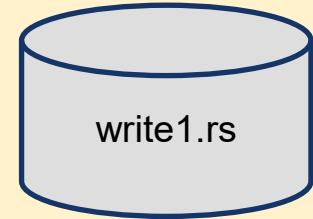
    for _ in 0..num_writes {
        // Scrivo i dati nel file e controllo il valore di ritorno
        match file.write(data) {
            Ok(bytes_written) => {
                total_bytes_written += bytes_written; // Aggiorno il conteggio totale dei byte scritti
            }
            Err(err) => {
                eprintln!("Errore durante la scrittura nel file: {}", err);
            }
        }
    }

    // Stampo il numero totale di byte scritti con successo nel file
    println!("Totale byte scritti nel file: {}", total_bytes_written);
    Ok(())
}

```

Text

se voglio solo ripetere una stessa azione
per 5 volte posso mettere l'underscore



la cosa fondamentale da ricordare
è che il metodo write ritorna un numero
di byte scritte che può non coincidere con
il valore che prende in input.
A differenza, il write_all assicura sempre di
scrivere i dati

```
use std::fs::File;
use std::io::{self, Write};

fn main() -> io::Result<()> {
    // Apro il file in modalità di scrittura
    let mut file = File::create("output.txt"?);

    // Dati da scrivere nel file
    let data = b"Hello, world!\n";

    // Scrivo i dati nel buffer del file
    file.write_all(data)?;

    // Eseguo il flush e gestisco il risultato
    match file.flush() {
        Ok(()) => println!("Dati scritti con successo nel file."),
        Err(err) => {
            eprintln!("Errore durante il flushing dei dati nel file: {}", err);
            return Err(err);
        }
    }

    Ok(())
}
```



write! ()

- La macro `write! ()` viene utilizzata per scrivere in un buffer dati **formattati**.
- Come per altre macro che interagiscono con l'I/O è necessario gestire eventuali errori che possono verificarsi durante l'operazione di scrittura.

```
use std::fs::File;
use std::io::{self, Write};

fn main() -> io::Result<()> {
    let mut buffer:Vec<u8> = Vec::new(); // Creazione di un buffer vuoto
    let num = 1;

    // Scrive su un buffer utilizzando la macro write!
    write!(buffer, "This is a test ");
    write!(buffer, "of the write! macro.\nNumero: {}\n", num)?;
    println!("Contenuto del buffer: {:?}", buffer);

    let path = "myfile";
    let mut output = File::create(path)?;
    write!(output, "{}", String::from_utf8_lossy(&buffer))?;

    Ok(())
}
```



la macro `write!()`
è un metodo alternativo
per scrivere su un file

flush

slide poco importante

- L'esecuzione della macro `write!()` include un flush, ossia garantisce che, prima di tornare, chiede al Sistema Operativo di salvare i dati
- Il Sistema Operativo, salva nel Kernel il buffer, con l'intenzione di scriverlo realmente sul file system successivamente (a sua discrezione, cercando di ottimizzare questa operazione quando il buffer raggiunge una dimensione opportuna)
- `write!()` garantisce che viene effettuato il flush.

std::io::Seek

- Obiettivo: trattare il file come fosse un grosso array
- Tratto che **permette di ri-posizionare il cursore di lettura/scrittura in un flusso di byte**
 - Quando il flusso attinge ad un dato di dimensione nota, è possibile posizionare il **cursore** (u64) in modo relativo rispetto all'inizio del flusso (**SeekFrom::Start(n: u64)**), alla sua fine (**SeekFrom::End(n: i64)**) o alla posizione corrente (**SeekFrom::Current(n: i64)**)
- Il tratto offre i seguenti metodi
 - **fn seek(&mut self, pos: SeekFrom) -> Result<u64>**: posiziona il cursore alla posizione (in byte) indicata dal parametro pos
 - **fn stream_position(&mut self) -> Result<u64>**: restituisce la posizione corrente del cursore rispetto all'inizio del flusso
 - **fn rewind(&mut self) -> Result<()>**: posiziona il **cursore** all'inizio del flusso

```
fn main() -> io::Result<()> {  
    // Apro il file in modalità di scrittura e lettura  
    let mut file = OpenOptions::new()  
        .read(true)  
        .write(true)  
        .create(true)  
        .open("example.txt")?;  
    file.write_all(b"Hello, world!")?;  
    // Sposto il cursore di lettura/scrittura alla fine del file  
    file.seek(SeekFrom::End(0))?;  
  
    // Scrivo dei dati aggiuntivi alla fine del file  
    file.write_all(b" Additional data")?;  
  
    // Sposto il cursore di lettura/scrittura alla posizione 7 nel file  
    file.seek(SeekFrom::Start(7))?;  
  
    // Scrivo dei dati in una posizione specifica nel file  
    file.write_all(b"Rust ")?;  
  
    // Sposto il cursore di lettura/scrittura all'inizio del file  
    file.seek(SeekFrom::Start(0))?;  
    let mut buffer = String::new();  
    file.read_to_string(&mut buffer)?;  
    println!("Contenuto del file: {}", buffer);  
    Ok()  
}
```



```

fn main() -> io::Result<()> {
    let mut file = OpenOptions::new().read(true).write(true).create(true).open("example.txt"?);

    file.write_all(b"Prova di Testo del File"?);
    file.seek(SeekFrom::Start(0))?;           // Sposto il cursore all'inizio del file utilizzando seek_from_start
    println!("Posizione corrente del cursore: {}", file.stream_position()?);
    let mut buffer = [0; 10];
    file.read_exact(&mut buffer)?;           // Leggo i primi 10 byte dal file
    println!("I primi 10 byte del file: {:?}", buffer);
    println!("Corrispondono a {:?}", String::from_utf8_lossy(&buffer));
    println!("Posizione corrente del cursore: {}", file.stream_position()?);
    file.seek(SeekFrom::End(0))?;             // Sposto il cursore alla fine del file utilizzando seek_from_end
    println!("Alla Fine: posizione corrente del cursore: {}", file.stream_position()?);
    file.seek(SeekFrom::Current(-5))?;        // Sposto il cursore 5 byte indietro dalla fine del file
    println!("Indietro di 5: posizione corrente del cursore: {}", file.stream_position()?);
    file.write_all(b" Additional data"?); // Scrivo ulteriori dati alla fine del file
    file.seek(SeekFrom::Start(10))?;         // Riporto il cursore alla posizione 10 all'interno del file
    println!("Vado in posizione 10: posizione corrente del cursore: {}", file.stream_position()?);
    let mut buffer = [0; 5];
    file.read_exact(&mut buffer)?;           // Leggo 5 byte dalla posizione corrente
    println!("Dati letti dalla posizione corrente: {:?}", String::from_utf8_lossy(&buffer));
    file.seek(SeekFrom::Start(0))?;
    let mut buffer = Vec::new();
    file.read_to_end(&mut buffer)?;
    println!("Tutto il file: {}", String::from_utf8_lossy(&buffer));
    Ok(())
}

```



1h 10' 40"

```
use std::io;
use std::io::{Cursor, Seek, SeekFrom, Read};
fn main() -> io::Result<()> {
    // Crea un buffer in memoria (simulando un file o uno stream)
    let data: Vec<u8> = b"Hello, Rust!".to_vec();
    let mut cursor = Cursor::new(data);

    let mut buffer = [0; 5];
    let bytes_read = cursor.read(&mut buffer)?; // Leggi una parte dei dati
    println!("Letti {} bytes: {:?}", bytes_read, &buffer[..bytes_read]);

    let current_position = cursor.stream_position()?; // Ottieni la posizione corrente del cursore
    println!("Posizione corrente del cursore: {}", current_position);

    cursor.rewind()?; // "Riavvolgi" il cursore all'inizio dello stream
    println!("Cursore riavvolto all'inizio.");

    let mut buffer_again = [0; 5]; // Leggi di nuovo dall'inizio
    let bytes_read_again = cursor.read(&mut buffer_again)?;
    println!("Letti di nuovo {} bytes: {:?}", bytes_read_again, &buffer_again[..bytes_read_again]);

    // Puoi anche usare seek per tornare all'inizio, ma rewind è più conciso per questo scopo specifico
    cursor.seek(SeekFrom::Start(0))?;
    println!("Cursore riportato all'inizio usando seek.");

    let mut buffer_third = [0; 5];
    let bytes_read_third = cursor.read(&mut buffer_third)?;
    println!("Letti per la terza volta {} bytes: {:?}", bytes_read_third, &buffer_third[..bytes_read_third]);

    Ok(())
}
```

Cursor, tipo predefinito che implementa i tratti Read, Write e Seek

alla slide n dicevamo che il tratto Read e altri possono essere usati anche sui vettori. Infatti in questo codice vediamo un esempio di applicazione in cui si utilizza il tipo Cursor



Text

Lettura di un file contenente dati binari

```
use std::fs::File;  
use std::io::Read;
```

codice eseguibile solo su linux

```
fn main() -> std::io::Result<()> {  
    // "/dev/urandom": file speciale, sorgente di numeri pseudo-casuali  
    // provenienti da una sorgente sicura e gestito dal kernel  
    let mut f = File::open("/dev/urandom"?);  
    let mut count = 0;  
    let mut buff = [0;4]; // buffer in cui depositare i byte  
    loop {  
        f.read_exact(&mut buff)?; // lettura del file  
        count += 1;  
        println!("{:?}",buff);  
        if buff.iter().any(|b| *b==0) {  
            return Ok(());  
        }  
        let i = i32::from_be_bytes(buff);  
        println!("{}",{:x}",count, i);  
    }  
}
```



```
// conversione del buffer in i32  
// uso il valore letto
```

Come si chiude un file?

- L'oggetto File implementa il pattern RAI che garantisce che la distruzione dell'oggetto **comporta il rilascio complessivo delle risorse che possiede, legate alla *handle* del Sistema Operativo**
- Il Sistema Operativo **ha allocato nel Kernel una struttura dati che è referenziata attraverso la handle del file**
- La chiusura di un file determina il rilascio di queste risorse allocate nel Kernel
- In Rust il metodo `close` non c'è perché la chiusura avviene all'atto della distruzione dell'oggetto File alla fine dello scope, in cui viene invocato il metodo `drop` che invoca la chiamata di `close` del Sistema Operativo.

Lettura e scrittura di contenuti strutturati

- Sebbene sia possibile operare a livello di singoli byte/caratteri e ricostruire un dato strutturato a partire dai suoi componenti elementari, spesso è poco conveniente farlo attraverso librerie predefinite
 - Rust mette a disposizione il framework **Serde** il cui scopo è generare implementazioni efficienti di funzioni di serializzazione e deserializzazione di strutture dati arbitrarie verso formati standard quali **JSON**, **CSV**, **Avro**, **BSON**, **YML** e altri
 - Tale framework definisce i tratti **serde::Serialize** e **serde::Deserialize** che possono essere implementati dai tipi definiti all'interno di un programma
- Serde offre un'implementazione predefinita per la serializzazione dei tipi elementari e di alcuni tipi della libreria standard
 - **String**, **&str**, **Vec<T>**, **HashMap<K,V>**, ...
 - Supporta inoltre la macro **#[derive(Serialize, Deserialize)]** per generare, in fase di compilazione, l'implementazione dei tratti corrispondenti, a condizione che la struttura dati cui tale macro è applicata non contenga elementi "patologici"

Uso del framework Serde

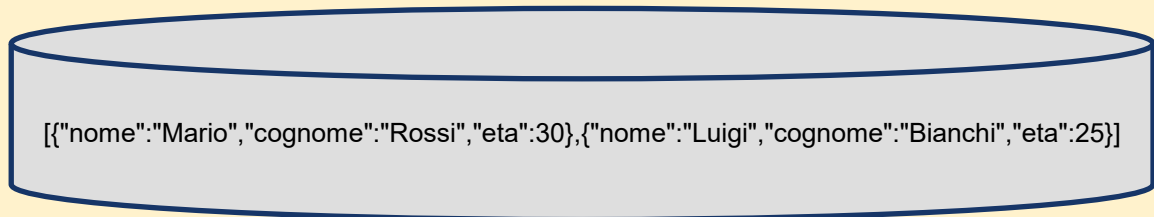
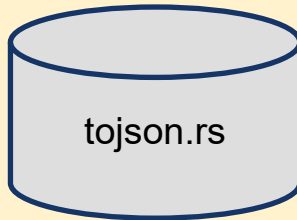
- Si inseriscono, nel file `cargo.toml`, le dipendenze
 - `serde = { version = "1.0", features = ["derive"] }`
 - `serde_json = "1.0"`
- La seconda indica il tipo di formato (specifico) da generare/leggere
 - In alternativa, è possibile indicare un altro sotto-progetto compatibile come `csv = "1.3"` o `bson = "2.9"`
- Si decora la struttura dati da leggere con la marco `#[derive(...)]`

```
#[derive(Serialize, Deserialize, Debug)]  
struct Data {  
    name: String,  
    data: Vec<u8>,  
    attributes: HashMap<String, String>,  
}
```

```
use serde::{Serialize, Deserialize};
use std::fs::File;
use std::io::Write;
// Definiamo una struttura dati per i nostri dati JSON.
#[derive(Debug, Serialize, Deserialize)]
struct Persona {
    nome: String,
    cognome: String,
    eta: u32,
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let persona1 = Persona {
        nome: "Mario".to_string(),
        cognome: "Rossi".to_string(),
        eta: 30,
    };
    let persona2 = Persona {
        nome: "Luigi".to_string(),
        cognome: "Bianchi".to_string(),
        eta: 25,
    };
    let persone = vec![persona1, persona2];

    // Serializziamo il vettore in formato JSON.
    let json_data = serde_json::to_string(&persone)?;
    // let json_data = serde_json::to_string_pretty(&persone)?;
    let mut file = File::create("persone.json")?; // Scriviamo il JSON su un file.
    file.write_all(json_data.as_bytes())?;
    Ok(())
}
```



```

use serde::Deserialize;
use std::error::Error;
use std::fs::File;
use std::io::BufReader;

#[derive(Debug, Deserialize)]
struct Person {
    name: String,
    age: u32,
    city: String,
}

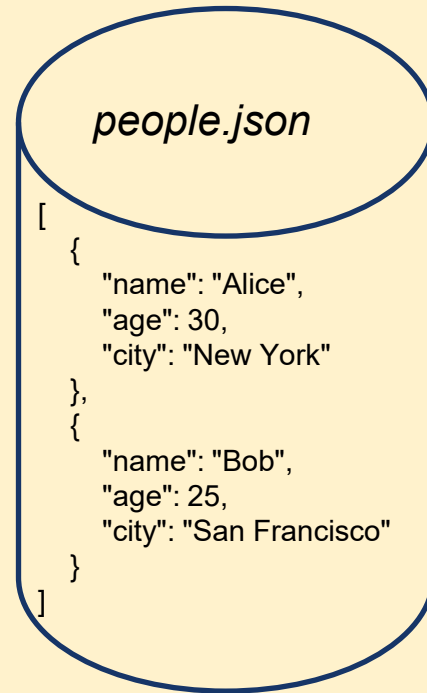
fn main() -> Result<(), Box<dyn Error>> {
    // Lettura del file JSON
    let file = File::open("people.json");
    let reader = BufReader::new(file);

    // Deserializzazione dei dati JSON in una struttura Rust
    let people: Vec<Person> = serde_json::from_reader(reader)?;

    // Stampa dei dati deserializzati
    println!("People:");
    for person in &people {
        println!("{:?}", person);
    }

    Ok(())
}

```



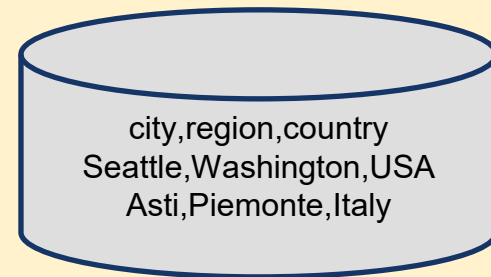
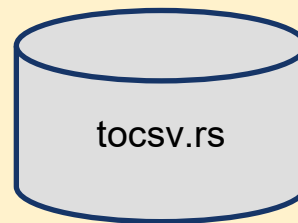
```

use std::error::Error;
use serde::{Deserialize, Serialize};
#[derive(Debug, Deserialize, Serialize)]
struct Record {
    city: String,
    region: String,
    country: String,
}

fn main() -> Result<(), Box<dyn Error>> {
    let records = vec![
        Record {
            city: "Seattle".to_string(),
            region: "Washington".to_string(),
            country: "USA".to_string(),
        },
        Record {
            city: "Asti".to_string(),
            region: "Piemonte".to_string(),
            country: "Italy".to_string(),
        },
    ];

    let file = std::fs::File::create("output.csv")?;
    let mut wtr = csv::Writer::from_writer(file);
    // Serializza ogni record nella struttura dati in un record CSV.
    for record in records {
        wtr.serialize(record)?;
    }
    wtr.flush()?; // Assicuriamoci che tutti i dati siano scritti nel file.
    Ok(())
}

```



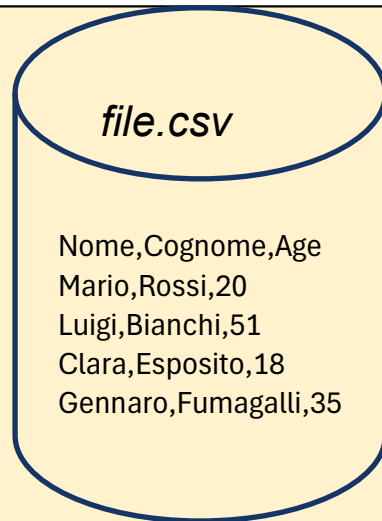
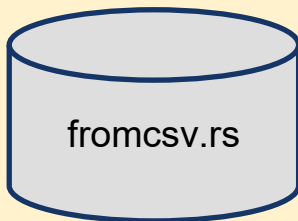
```
use std::{error::Error, fs::File};
use serde::Deserialize;
```

```
#[derive(Debug, Deserialize)]
struct Persona {
    Nome: String,
    Cognome: String,
    Age: u32,
}
```

```
fn main() -> Result<(), Box<dyn Error>> {
    // Apri il file CSV in lettura.
    let file = File::open("./file.csv")?;
    let mut rdr = csv::Reader::from_reader(file);

    // Leggi ogni record dal file CSV e deserializzalo in una struttura Persona.
    for result in rdr.deserialize() {
        let persona: Persona = result?;
        println!("{:?}", persona);
    }

    Ok(())
}
```



Glossario

- File: blocco di dati identificato da un nome e memorizzato su un dispositivo di memorizzazione persistente
- Directory (cartella): contenitore virtuale all'interno del file system utilizzato per raggruppare file e altre directory
- File system: struttura logica che un sistema operativo utilizza per organizzare, memorizzare e gestire i file e le directory su un dispositivo di memorizzazione persistente
- Path: stringa di caratteri che specifica la posizione univoca di un file o di una cartella all'interno del file system
- Collegamento simbolico (symlink): un particolare tipo di file che altro non è che un rimando ad un altro file o directory
- Handle: identificatore astratto che il sistema operativo assegna ad un programma quando il programma chiede di accedere ad un file. Una volta ottenuto un handle, il programma può utilizzare questo identificatore per eseguire le varie operazioni sul file (lettura, scrittura, spostamento del cursore, chiusura)
- Cursore (o puntatore a file): indicatore che specifica la posizione all'interno del file da cui la prossima operazione di lettura o scrittura avrà luogo
- Chunk: blocco contiguo di dati

Glossario

- Flush: operazione che forza l'invio dei dati memorizzati in memoria (buffer) al disco, garantendo così che le modifiche vengano salvate definitivamente
- Seek: operazione che permette di modificare esplicitamente la posizione del cursore all'interno di un file, senza dover scandire tutti i dati precedenti
- Random access (accesso non sequenziale): capacità di un programma di accedere direttamente a qualsiasi posizione specifica all'interno del file, senza dover scorrere sequenzialmente tutti i dati precedenti.
- Serializzazione: operazione effettuata per salvare lo stato di una struttura dati (interna) in un formato che può essere facilmente memorizzato su file oppure trasmesso attraverso la rete, ossia è il processo di conversione di una struttura dati complessa in una sequenza lineare di byte.
- Deserializzazione: processo inverso, rispetto alla serializzazione, ossia è il processo che permette di generare una struttura dati complessa a partire da una sequenza lineare di byte (ad es. un file).